Computer Architecture and System Software

Chapter 01: Course Introduction and Overview of Computer Systems

Instructor:
David Bosc
Applied Computer Science
University of Winnipeg

2

Preliminaries

Instructor

- David Bosc
  - E-mail: da.bosc@uwinnipeg.ca

- Background:
  - Video Games Industry (Tools, Gameplay, UI and Online programming)

Website

- Check the Nexus page regularly
- Updates will be posted weekly
  - Course announcements
  - Syllabus available for download

Syllabus

- The course syllabus can be downloaded from the course website

LLM Prompt Examples

- Use them to write a study guide, practice tests
  - Writing one yourself first, then compare LLM's guide.
  - Fill in extra info is a great way to understand the theory.
  - Don't understand one of the points on your study guide? Ask for a detailed explanation!

LLM Prompt Examples (cont)

ChatGPT

Lecture 2.pdf
PDF

Write me a study guide based off the attached lecture, skipping the announcement slide.

2/2

Here's a study guide based on the lecture, excluding the announcement slide.

Computer Systems Study Guide

Answer in chat instead

I've compiled a structured study guide based on the lecture content, skipping the announcements. Let me know if you need any modifications or additional details!

Computer Systems Study Guide

Study Guide: Overview of Computer Systems & Chapter 2

1. Representing and Manipulating Information

- **Bits and Context:** All information in a system is represented as bits, which must be encoded numerically and stored in binary.

2. Number Systems

- **Decimal System:** Base-10 system using digits 0-9.
- **Binary System:** Base-2 system using only 0s and 1s. Used in computing for easier storage, transmission, and logic implementation.
- **Hexadecimal System:** Base-16 system (digits 0-9 and A-F). More compact representation of binary numbers.

3. Conversions

- **Binary to Hex:** Group binary digits into nibbles (4-bit groups) and convert each to hex.

LLM Prompt Examples (cont)

Write me some possible test questions that could be given based off this lecture

Here are some possible test questions based on the lecture:

1. Explain why computers use binary instead of decimal number systems.
2. Convert the binary number 101011 to hexadecimal.
3. Given a 32-bit address bus, how much memory can be addressed?
4. Differentiate between big-endian and little-endian memory storage.

LLM Prompt Examples (cont)

- Completed the worksheets? Use them to make more practice problems, or variants of the labs!

ChatGPT

Worksheet1.pdf
PDF

Create a worksheet like the one attached, with different questions and detailed solutions

Here is a new worksheet with different questions and detailed solutions based on the same format:

Number Conversion Ws

Answer in chat instead >

I have created a new worksheet with different questions and detailed solutions. Let me know if you need any modifications or additional questions!

Number Conversion Ws

1. Convert the following binary numbers to decimal/hexadecimal:

a. $(101110101011)_2$
b. $(1001101)_2$
c. $(110101011)_2$
d. $(10100010)_2$

Solutions:

a. $(101110101011)_2 = (2987)_{10} = (BAD)_{16}$
b. $(1001101)_2 = (77)_{10} = (4D)_{16}$
c. $(110101011)_2 = (427)_{10} = (1AB)_{16}$
d. $(10100010)_2 = (162)_{10} = (A2)_{16}$

2. Convert the following hexadecimal numbers to binary/decimal:

a. $(3F2)_{16}$
b. $(A7)_{16}$
c. $(D4B)_{16}$
d. $(1C)_{16}$

Classroom Procedure

- Be on time, and stay until the end
- Ask questions anytime
- Typically, we will take a break at 7:15 and restart at 7:30

Lecture Format

- Lectures will consist of
  - Announcements
  - Slides (available from the course website)
  - Problems and examples solved in class on the board
    - Attendance for solutions is mandatory!
  - In-class demos for assembly programming

Lab Format

- Will be led by a demonstrator
- You will be provided the lab material Friday morning on Nexus under **Assignments**. You will have until 11:59pm of Friday to submit it.
- **Important:** It is very important to attend labs to get practice with assembly programming. You will not be successful unless you put the time in.
- Questions will **only** be addressed in lab, by the lab demonstrator of your **enrolled section**.

Outline

- Course theme
- Preliminaries
- A Tour of Computing Systems
  - Information is Bits + Context
  - Compilation System
  - Hardware Organization
  - The Operation System

Course Theme

Course Textbook

- R. E. Bryant and D. R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd Edition, Upper Saddle River, NJ, USA: Pearson/Prentice Hall, 2011.
- Textbook website: http://csapp.cs.cmu.edu/

Course Theme

- Most Systems Courses are Builder-Centric
- This course is programmer-centric
  - Purpose to show how by knowing more about the underlying system, one can be more effective as a programmer
- The goal of this course is to give you a programmer-centric view of the "what is going on under the hood of a computer system"
- Need to understand details of underlying implementations

Course Theme

- Enable you to:
  - Write programs that are more reliable and efficient
  - Avoid strange numerical errors caused by representation of numbers
  - Optimize code by using clever tricks that exploit designs of modern systems
  - Example: Buffer overflows are sometimes exploited to bypass network security

Class Discussion

- What is the difference between *int* and *float* data types?

Course Theme

- An int is not an integer, a float is not a real number
- Example 1) Is $x^2 >= 0$?
  - float: yes
  - int: no
    - 40000 * 40000 = 1600000000
    - 50000 * 50000 = ?
- Example 2) Is (x + y) + z = x + (y + z) ?
  - int: yes
  - float: no
    - (1e20 + -1e20) + 3.14 = ?
    - 1e20 +(-1e20 + 3.14) = ?

Class discussion

- What does a compiler do?

Course Theme

- You've got to know assembly language
  - Compilers are much better & more patient than you are
- But: Understanding assembly is key to machine-level execution model
  - Behaviour of programs in presence of bugs
    - High-level language model breaks down
  - Tuning program performance
    - Understand optimizations done/not done by the compiler
    - Understanding sources of program inefficiency
  - Implementing system software
    - Compiler has machine code as target
    - Operating systems must manage process state

Course Theme

Memory Matters

- Memory is not unbounded
  - It must be allocated and managed
  - Many applications are memory dominated
- Memory referencing bugs are often the most difficult to find
  - Effects are distant in both time and space
- Memory performance is not uniform
  - Cache and virtual memory effects can greatly affect program performance
  - Adapting program to characteristics of memory system can lead to major speed improvements

Course Theme

- Memory performance example

```
void copyij(int src[2048][2048],
            int dst[2048][2048])
{
  int i,j;
  for (i = 0; i < 2048; i++)
    for (j = 0; j < 2048; j++)
      dst[i][j] = src[i][j];
}
```

```
void copyji(int src[2048][2048],
            int dst[2048][2048])
{
  int i,j;
  for (j = 0; j < 2048; j++)
    for (i = 0; i < 2048; i++)
      dst[i][j] = src[i][j];
}
```

21 times slower
(Pentium 4)

Course Theme

Computers do more than execute programs

- They need to get data in and out
  - I/O system critical to program reliability and performance
- They communicate with each other over networks
  - Many system-level issues arise in presence of network
    - Concurrent operations by autonomous processes
    - Coping with unreliable media
    - Cross platform compatibility
    - Complex performance issues

Chapter 1

A Tour of Computer Systems

Information is bits + context

- Example:
  - What do the numbers below represent ?

35	105	110	99	108	117	100	101
32	60	115	116	100	105	111	46
104	62	10	10	105	110	116	32
109	97	105	110	40	41	10	123
10	32	32	32	32	112	114	105
110	116	102	40	34	104	101	108
108	111	44	32	119	111	114	108
100	92	110	34	41	59	10	125

Information is bits + context

- Answer: Depends on the context.
  - In this case they could represent text

#	i	n	c	l	u	d	e
35	105	110	99	108	117	100	101
<sp>	<	s	t	d	i	o	.
32	60	115	116	100	105	111	46
h	>	\n	\n	i	n	t	<sp>
104	62	10	10	105	110	116	32
m	a	i	n	(	)	\n	{
109	97	105	110	40	41	10	123
\n	<sp>	<sp>	<sp>	<sp>	p	r	i
10	32	32	32	32	112	114	105
n	t	f	(	"	h	e	l
110	116	102	40	34	104	101	108
l	o	,	<sp>	w	o	r	l
108	111	44	32	119	111	114	108
d	\	n	"	)	;	\n	}
100	92	110	34	41	59	10	125

- ASCII: American Standard Code for Information Interchange

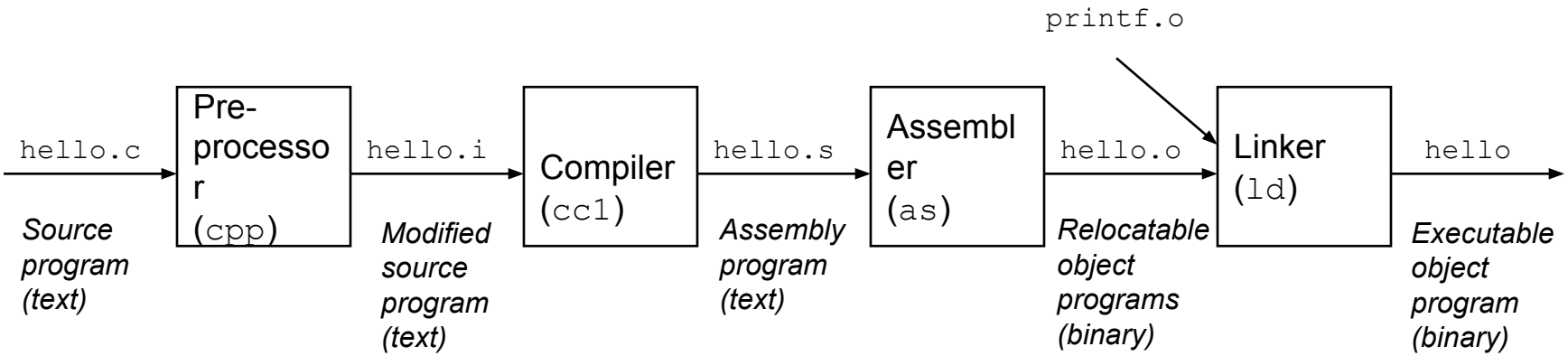
Information is bits + context

- All information in a system is represented as a bunch of bits!
- Problem: What do they represent?
- For example: 0100 1011
  - Integer 75
  - ASCII character K

Information is bits + context

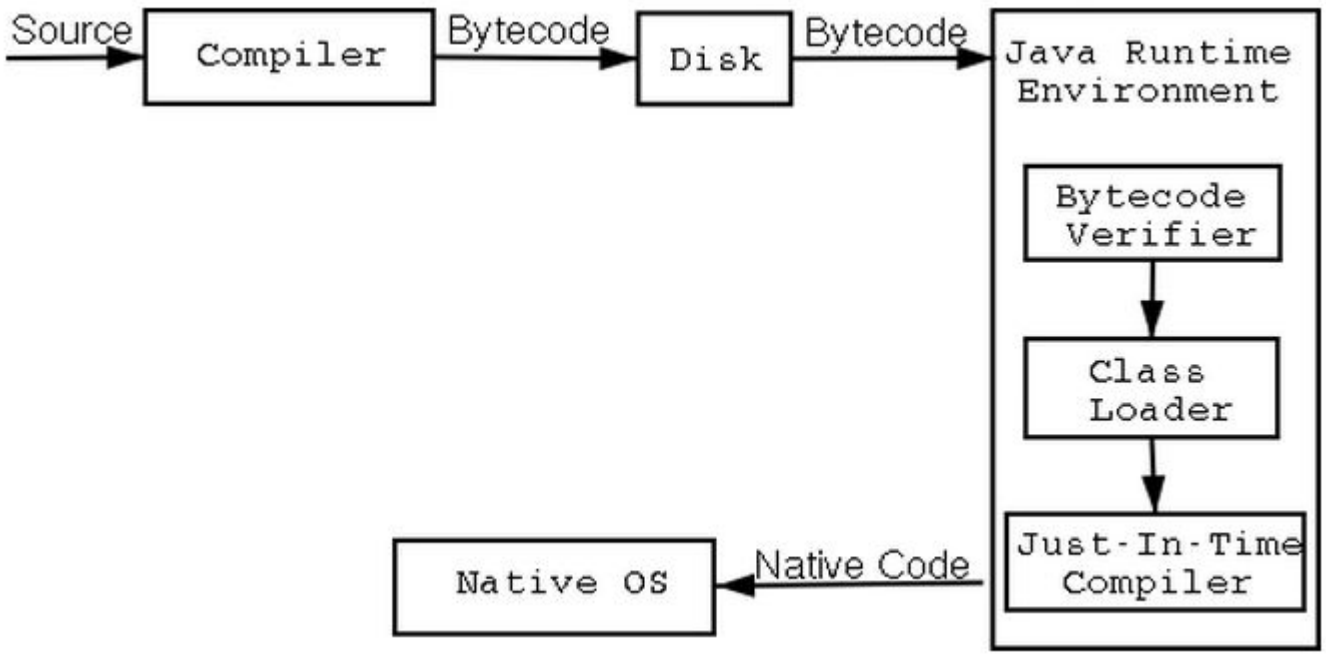
- Context is determined by:
  - Computer architecture
    - Big endian and little endian byte ordering
  - Operating system
    - Different OS use different markers to indicate the end of a line in a text file (CR and LF)
      - \r\n DOS\Windows
      - \n for Unix and Unix-like world
  - Application software
    - MS Word doc files

C/C++ Program Compilation



Java Compilation Process

- See http://en.wikipedia.org/wiki/Java_Virtual_Machine



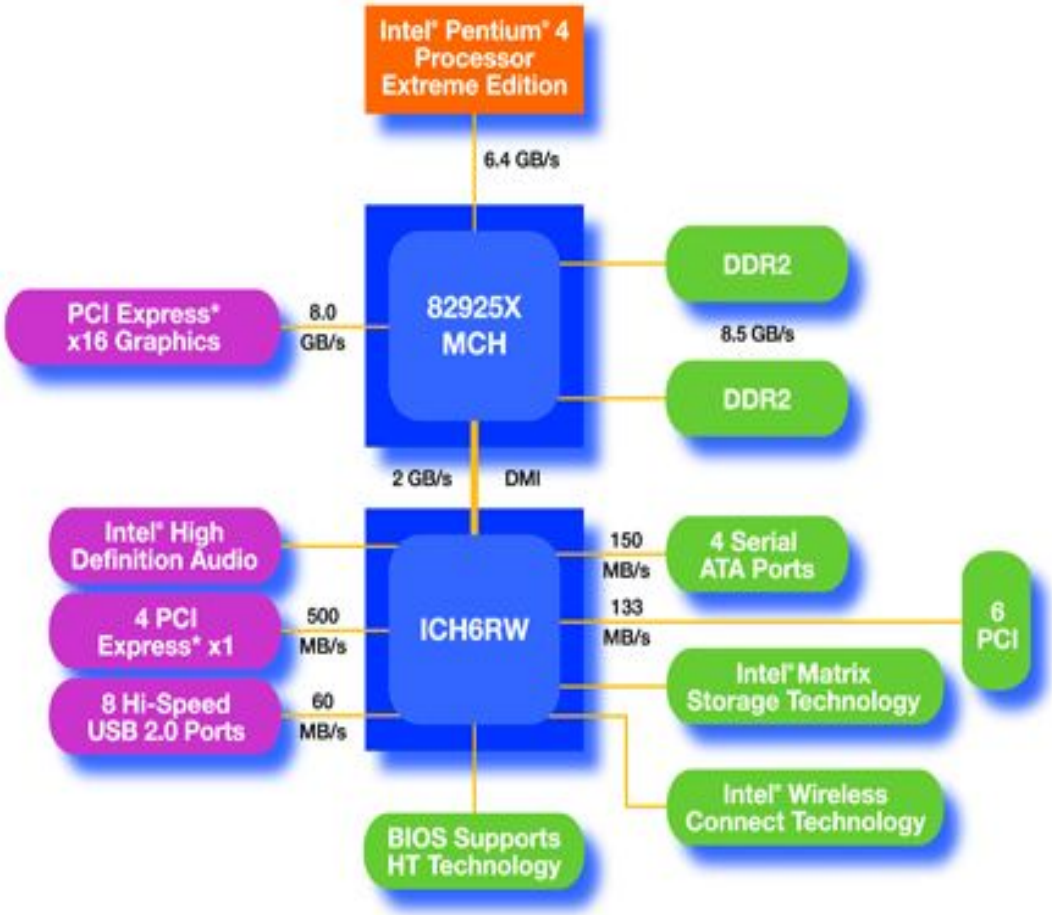
Program Compilation

- Important reasons to understand compilation process:
  - Optimizing system performance
    - Why does a loop run faster if we sum into a local variable instead of an argument that is passed by reference?
  - Understanding link-time errors
  - Avoiding security holes
    - What are buffer overflow bugs?

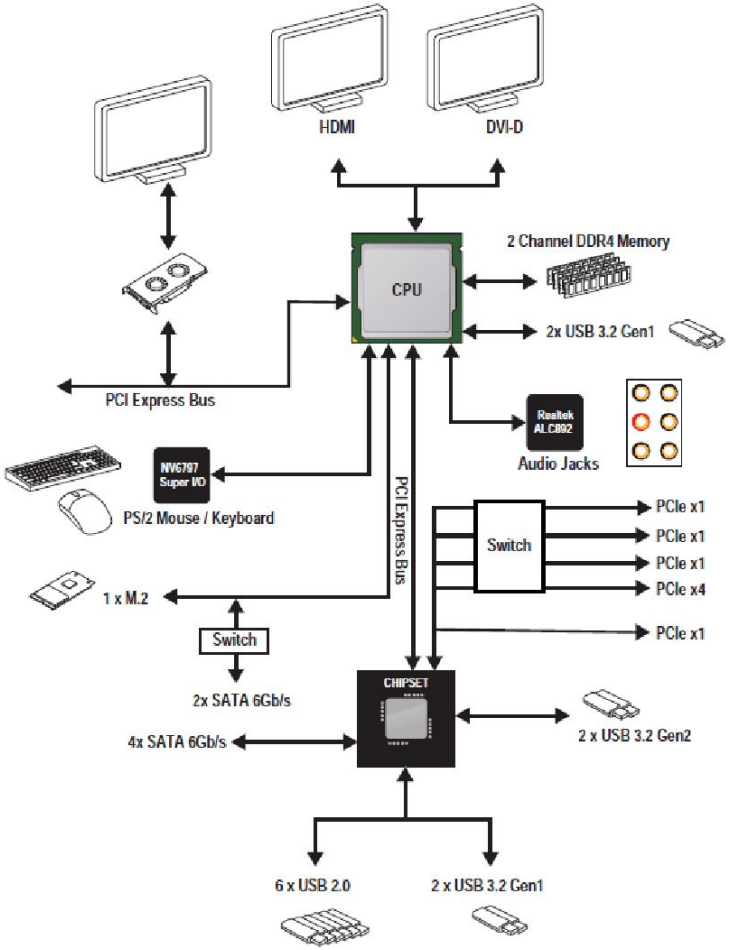
Program Execution

- How is an executable file executed ?
  - To answer this question we need to understand the hardware organization of a typical system
  - Also need to know the role of the operating system

Layout of a Real System

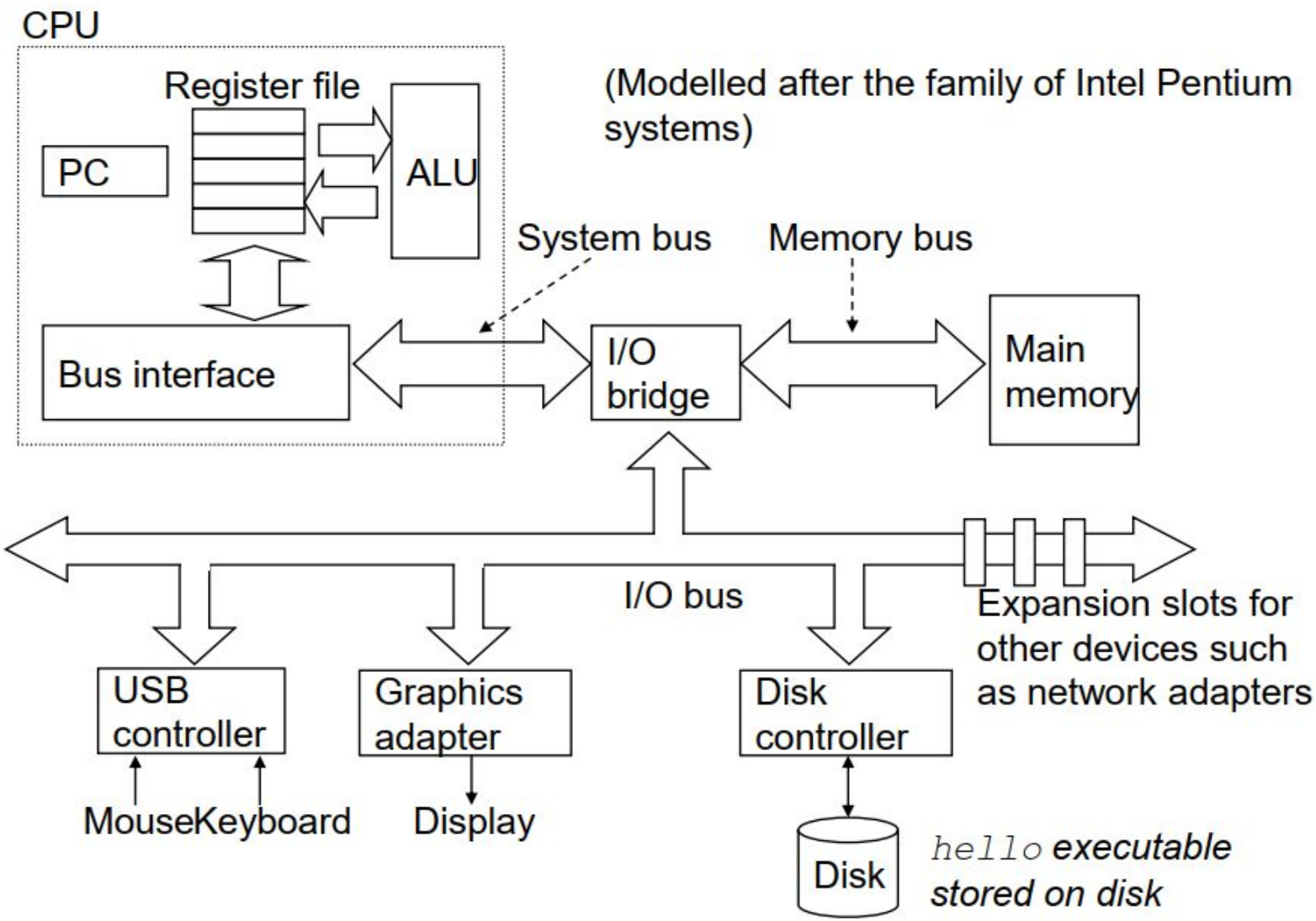


Layout of a Real System



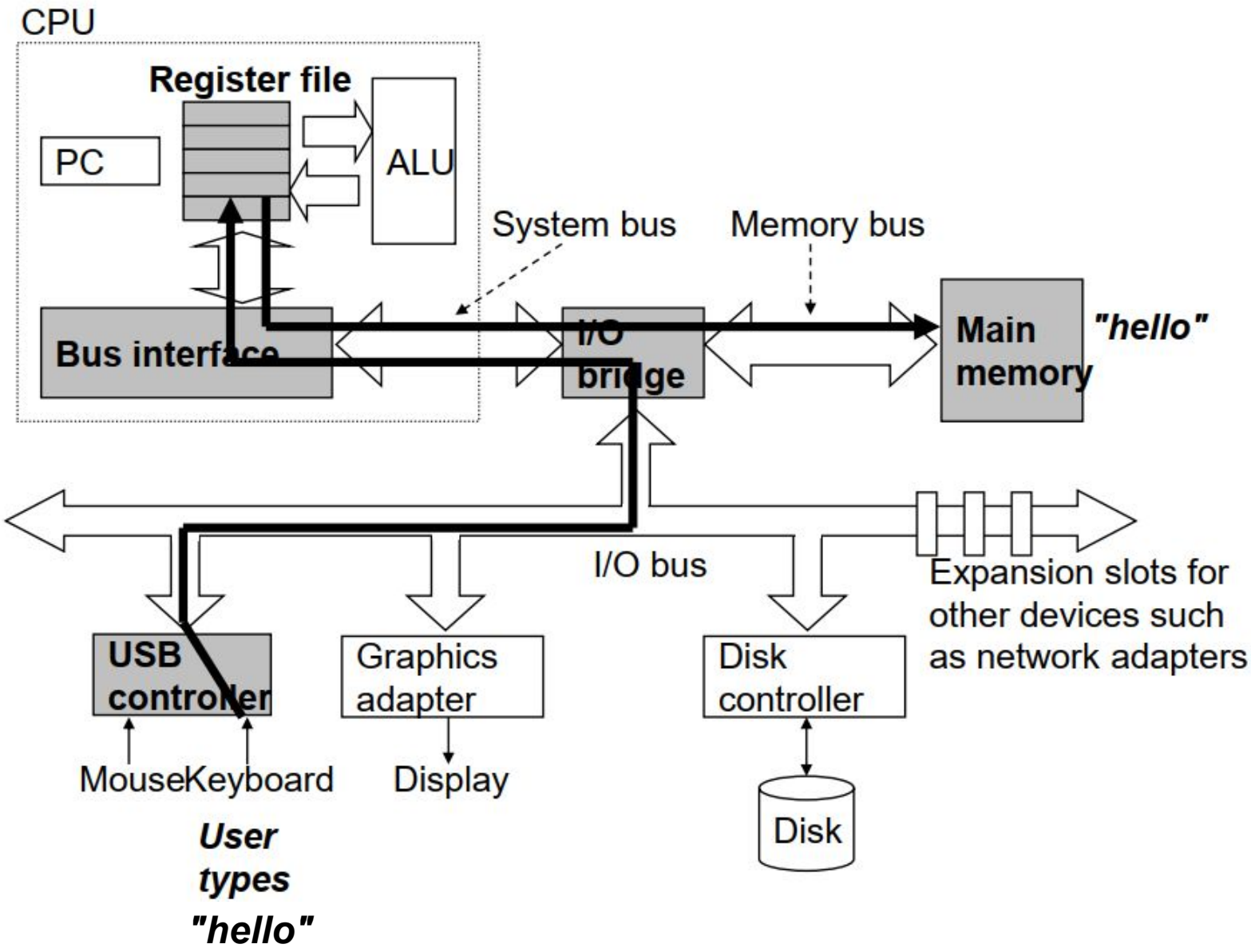
https://download.msi.com/archive/mnu_exe/mb/E7B86v1.2-GSE-LITE.pdf

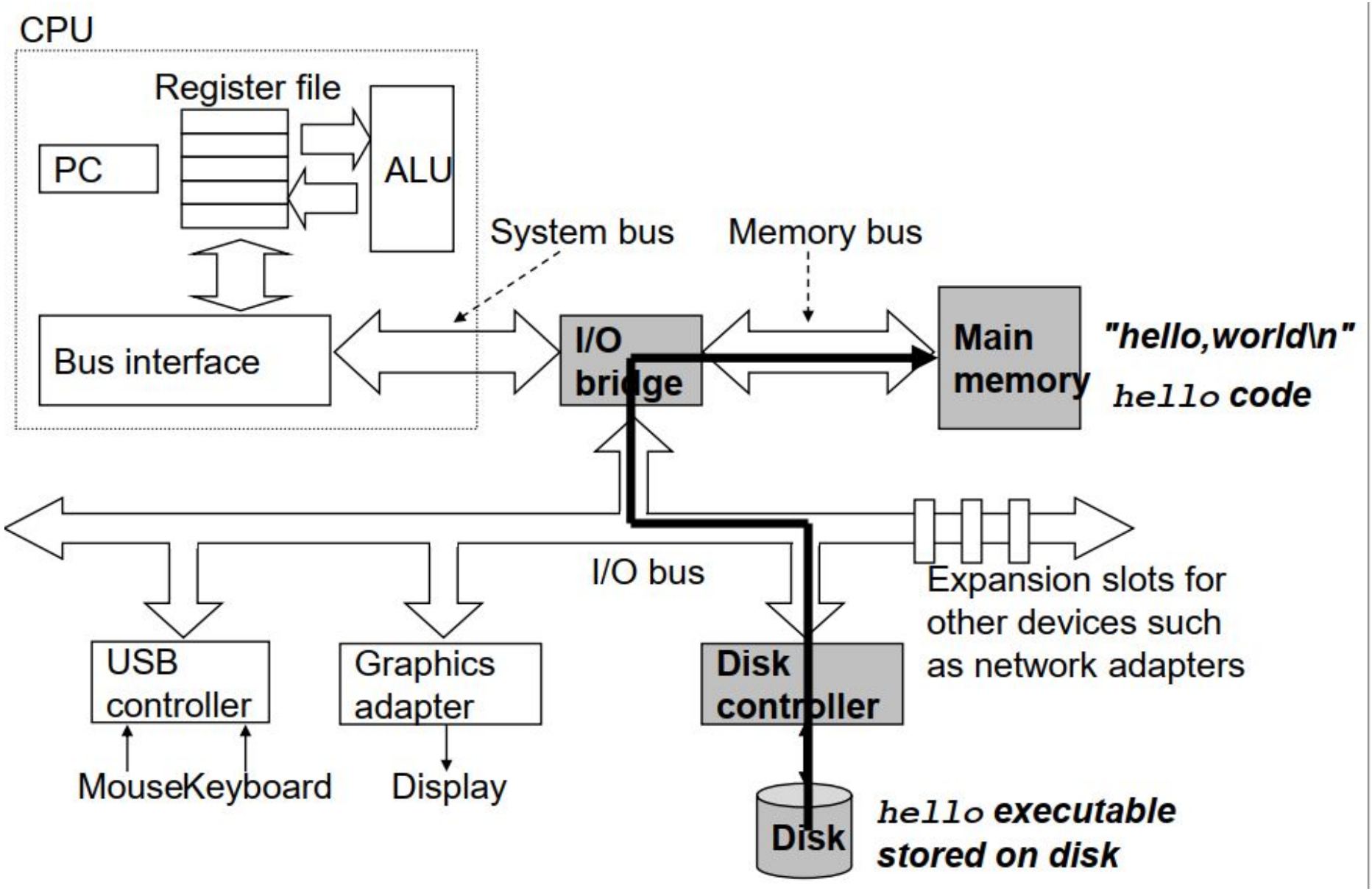
Hardware Organization

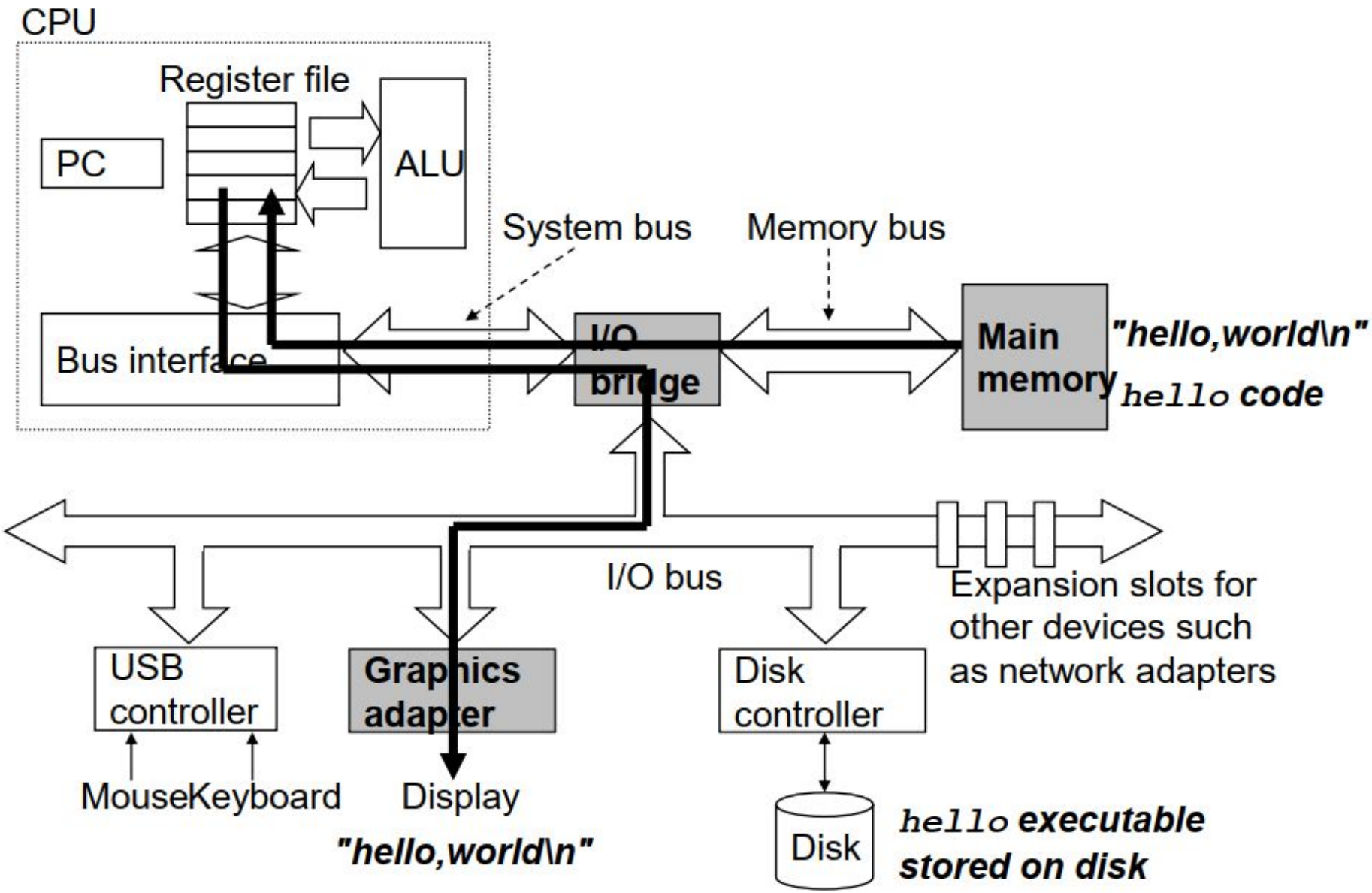


Running the *hello* program

- Referring to example on p. 10 of the text
  - Simple "Hello World" program
- To run program (majority of details omitted)
  - Load program into memory
  - Point PC to main routine of program
  - CPU executes instructions


CPU
Register file
PC
ALU
Bus interface
System bus
Memory bus
I/O bridge
Main memory
"hello"
I/O bus
Expansion slots for other devices such as network adapters
USB controller
Graphics adapter
Disk controller
Mouse
Keyboard
Display
Disk
User types "hello"


CPU
Register file
PC
ALU
Bus interface
System bus
Memory bus
I/O bridge
Main memory
"hello,world\n"
hello code
I/O bus
Expansion slots for other devices such as network adapters
USB controller
Graphics adapter
Disk controller
MouseKeyboard
Display
Disk
hello executable stored on disk


CPU
Register file
PC
ALU
Bus interface
System bus
Memory bus
I/O bridge
Main memory
"hello,world\n"
hello code
I/O bus
Expansion slots for other devices such as network adapters
USB controller
Graphics adapter
Disk controller
Mouse
Keyboard
Display
"hello,world\n"
Disk
hello executable stored on disk

Overhead

- Problem: A computer spends a lot of time copying data
- Hello program
  - Copied from disk to main memory to processor
- String "hello, world\n"
  - Copied from disk to main memory to display device
- Overhead slows down real work of processor
- Goal: make these copy instructions as fast as possible

Class Discussion

- How is information represented/managed at the hardware level?
  - What is a bit physically?
  - Do we know how long it takes to read a byte?

Storage Devices

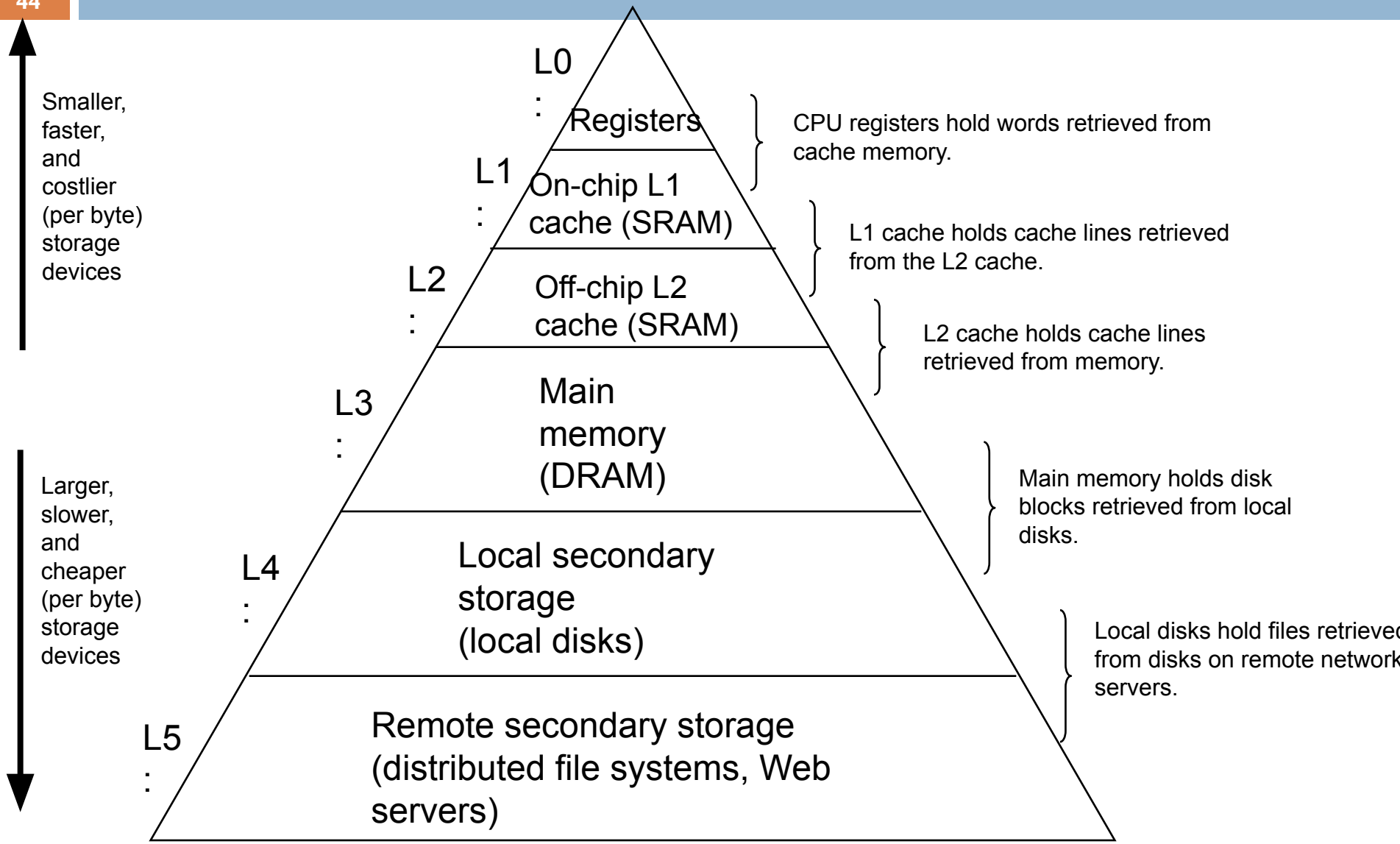
- Physical laws dictate storage device operation
  - Larger capacity storage devices are slower
  - Faster devices are more expensive
- Examples:
  - A disk drive (HD) might be 100 x larger than main memory
    - But, it might take CPU 10,000,000 x longer to read a word from disk than from memory
  - A register file stores only a few hundred bytes (compared to millions of bytes in memory)
    - However, CPU can read data from registers
      - 100 x faster than memory

http://www.mackintoshconsultants.co.uk/images/storage-devices.gif

http://upload.wikimedia.org/wikipedia/commons/8/87/PersonalStorageDevices.agr.jpg

http://upload.wikimedia.org/wikipedia/commons/c/ca/Memory_module_DDRAM_20-03-2006.jpg

Storage Hierarchy



Storage Hierarchy

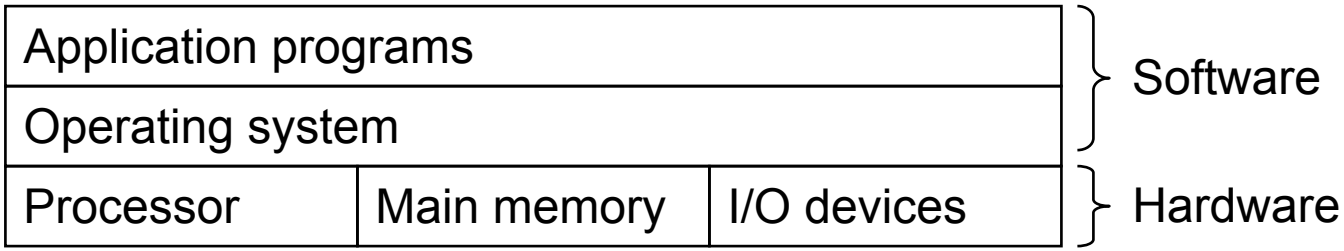
- Main Idea:
  - Storage at one level serves as a cache for storage at the next lower level
- Programs can run much faster if the programmer exploits caches effectively
- Problem: How to manage all the different memory and devices?

Class Discussion

- What does an operating system do? What are its main functions?

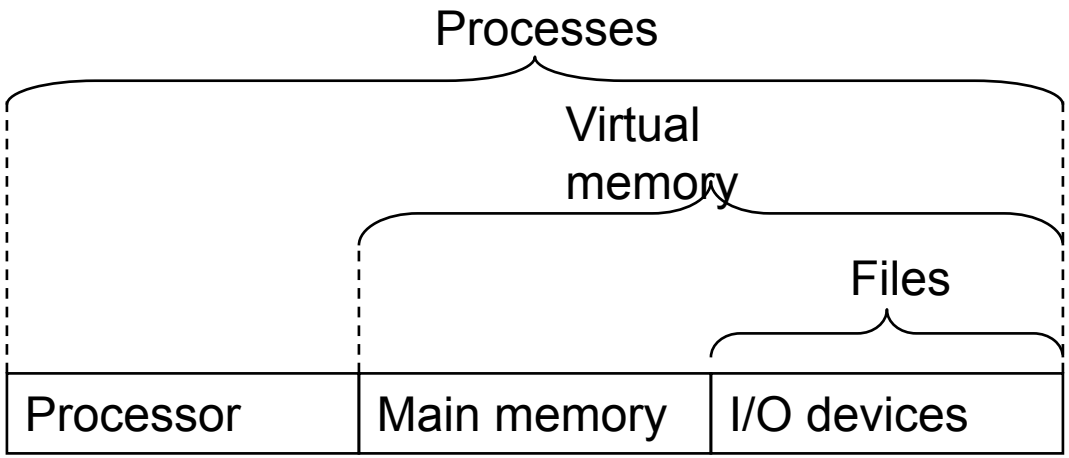
Operating System

- Hello and shell programs did not access the keyboard, display, disk, or main memory directly
- Instead, they relied on services provided by the Operating System (OS)
- OS is a layer of software between application programs and hardware
- All attempts to access the hardware must go through OS first



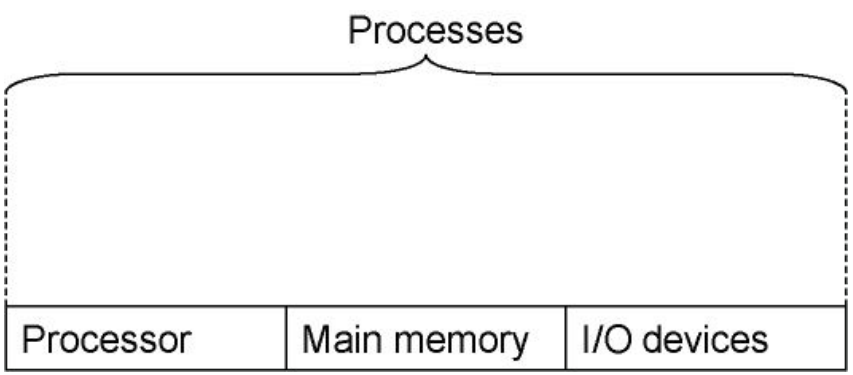
Operating System

- OS has 2 purposes:
  - Protect the hardware from misuse by applications
  - Provide applications with simple and uniform mechanisms for manipulating various complicated low-level hardware
- OS achieves both goals via the fundamental abstractions:
  - Processes
  - Virtual memory
  - Files



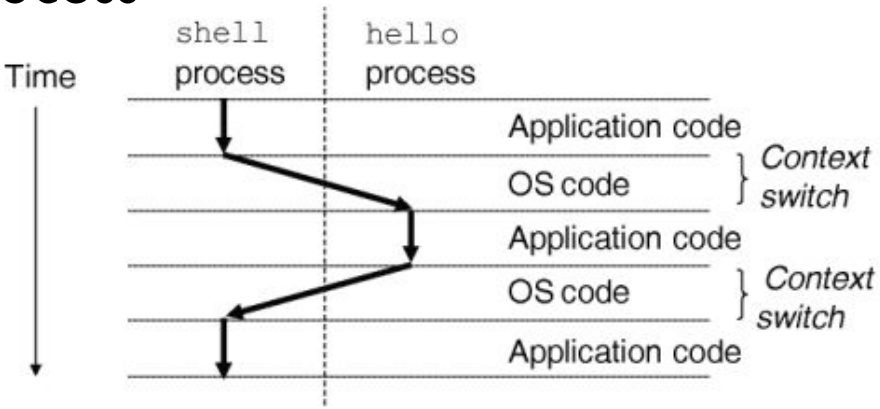
Processes

- Operating system provides illusion that program is the only one running
- Appears to:
  - Have exclusive use of processor, main memory, and I/O
  - Execute instructions one after the other without interruption
  - Only program's code and data reside in memory
- Illusions provided by processes
- A process is the OS abstraction for a running program



Context Switching

- Processes share the processor and system resources
- OS allocates time
- Switches context by
  - Saving context of current process (PC, register, memory)
  - Restoring context of previous process (PC, register, memory)
  - Passing control to new process

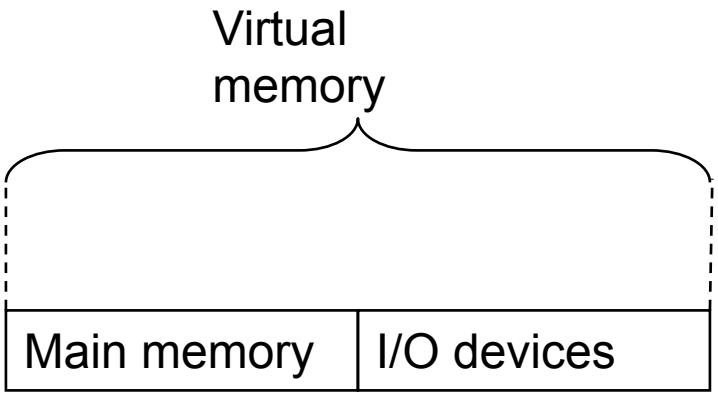


Threads

- Processes with multiple executions
  - A process can have more than one execution
  - Each execution is called a thread
- Threads execute concurrently as well
- Differences:
  - Process do not share memory
  - Threads view: it does not have the whole machine to itself

Virtual Memory (VM)

- VM: interaction of hw exceptions, hw address translation, main memory, disk files, and kernel software
- Goal: Provide each process with a large, uniform, and private memory address space

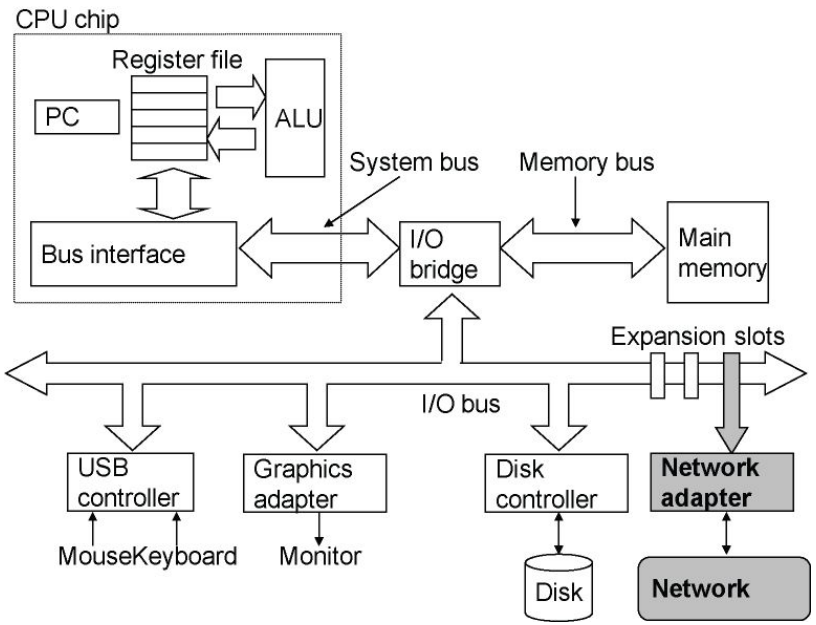
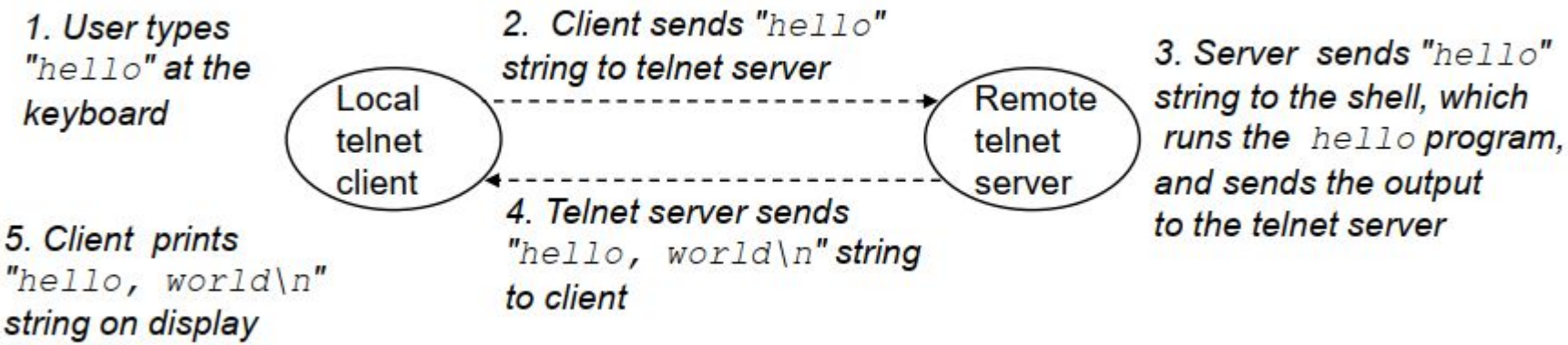


Files

- A file is a sequence of bits.
- Every I/O device is modeled as a file
  - Read/write: disks, networks
  - Read-only: keyboard, mouse, web-cam
  - Write-only: video, printer
- Advantages:
  - Uniform interface for varied I/O devices
  - Abstracts away all the underlying details on how data read/written
  - Programs written on one system can be easily ported to another

Networks

- Systems are often linked to other systems by networks
- Viewed as another I/O device
- Akin to reading/writing to a file



Remote Execution